Quantization in LLMs

A complete reference — RTN, GPTQ, AWQ, salient weights, and the activation story

1. The Problem Quantization Solves

A trained neural network is just a giant pile of numbers — its weights. Inference loads these weights into GPU memory and multiplies them with your input. The model file is the weights.

By default each weight is stored as a 32-bit float (FP32), which uses 4 bytes per weight. For a 70-billion-parameter model:

```
70,000,000,000 weights  ×  4 bytes  =  280 GB
```

That doesn't fit on any single consumer GPU. An H100 has 80 GB. A laptop has 16 GB. The model must shrink before it can run anywhere practical.

Quantization asks a simple question: do we really need 32 bits of precision per weight? What if 8 bits is enough? Or 4? Or 2?

Format	Bits per weight	Distinct values	Size of 70B model
FP32	32	~4 billion	280 GB
FP16	16	~65,000	140 GB
INT8	8	256	70 GB
INT4	4	16	35 GB
INT2	2	4	17.5 GB

> At INT4, a 70B model fits on a single 80 GB GPU. Same model architecture, same number of weights — just smaller numbers.

2. The Bucket Trick (How Quantization Actually Works)

This is the core mechanism every quantization method builds on.

Setup

Imagine you have 10 weights from a model layer:

```
0.7384, -0.2156,  0.5921, -0.8043,  0.1247,
0.6688, -0.4419,  0.3372, -0.0921,  0.7755
```

Each is currently a 32-bit float (4 bytes each → 40 bytes total).

Goal: store each one using only 4 bits (half a byte → 5 bytes total). 8× smaller.

Problem: 4 bits can only represent 16 distinct values (0 through 15). But our weights are decimals. How do we compress a decimal into a number 0–15?

Step 1: Find the range

Look at all 10 weights. Smallest is −0.8043, largest is +0.7755. All weights live somewhere in this range.

Step 2: Slice the range into 16 equal slots

```
slot 0:  -0.8043
slot 1:  -0.6990
slot 2:  -0.5937
slot 3:  -0.4884
slot 4:  -0.3831
slot 5:  -0.2778
slot 6:  -0.1725
slot 7:  -0.0672
slot 8:  +0.0381
slot 9:  +0.1434
slot 10: +0.2487
slot 11: +0.3540
slot 12: +0.4593
slot 13: +0.5646
slot 14: +0.6699
slot 15: +0.7755
```

Gap between consecutive slots is constant:

```
scale_factor = (0.7755 - (-0.8043)) / 15  =  0.1053
```

This scale factor is the size of one bucket — and it's the only extra information you need to keep beyond the slot numbers themselves.

Step 3: Snap each weight to its closest slot, store only the slot number

Original weight	Closest slot	Slot value	Slot # stored
0.7384	slot 15	0.7755	15
−0.2156	slot 6	−0.1725	6
0.5921	slot 13	0.5646	13
−0.8043	slot 0	−0.8043	0
0.1247	slot 9	0.1434	9
0.6688	slot 14	0.6699	14
−0.4419	slot 4	−0.3831	4
0.3372	slot 11	0.3540	11
−0.0921	slot 7	−0.0672	7
0.7755	slot 15	0.7755	15

Final stored data:

```
Slot numbers: [15, 6, 13, 0, 9, 14, 4, 11, 7, 15]   (4 bits each)
```

```
Scale factor: 0.1053                                    (one float for whole matrix)
Minimum:      -0.8043                                   (one float for whole matrix)
```

Step 4: Recover an approximate float when needed

```
recovered_value = slot_number × scale_factor + minimum

For slot 15:
   15 × 0.1053 + (-0.8043) = 0.7752

Original was 0.7384. Recovered is 0.7752. Off by 0.0368.
```

> This is the fundamental trade — lose precision, save space. The model still runs, but every weight comes back slightly off. The art of quantization is making sure those small errors don't compound into noticeable accuracy loss.

The mental picture

Think of a ruler. Original floats are points anywhere along a smooth ruler. Quantization replaces it with a ruler that has only 16 tick marks. Every point snaps to its nearest tick. You lose the in-between positions, but the ruler can now be described with just 4 bits per point.

Use 8 bits → 256 ticks → smaller rounding error, bigger file. That's the dial you turn.

3. Activation Values — The Missing Piece

Before we discuss the smarter quantization methods (GPTQ, AWQ), one term must be crystal clear: activation values. The word 'activation' is overloaded in deep learning and trips everyone up.

Two meanings of 'activation'

Term	What it means
Activation function	The non-linear function (ReLU, GeLU, sigmoid) that squashes values inside a neuron. Static formula.
Activation value	The actual numerical output that flows out of one layer and into the next at runtime. Dynamic — depends on input.

In quantization discussions, 'activations' always means the second one — the live numbers traveling between layers when the model runs.

Weights vs Activations

Weights	Activations
Static — stored in the model file	Dynamic — created fresh on every inference
Don't change at inference time	Change with every different input
Locked in during training	Computed layer by layer at runtime

Weights	Activations
What we quantize aggressively	Often left in higher precision

How a layer computes its output

Inside any layer, a single neuron's computation looks like:

```
neuron_output = w₁·a₁ + w₂·a₂ + w₃·a₃ + ... + wₙ·aₙ

where:
   wᵢ = weights (loaded from model file, static)
   aᵢ = activation values (numbers flowing in from previous layer, dynamic)
```

Each term is a (weight × activation) product. The neuron sums these up, then applies an activation function. That's it.

4. Salient Weights — What They Are and Why They Matter

Definition

A salient weight is one whose rounding error has an outsized effect on the model's output. A non-salient weight is one you can round sloppily and the model barely notices.

Salience is not about the weight's own magnitude. It's about the size of the activation it gets multiplied with.

Worked example

Suppose layer 5 receives this incoming activation vector:

```
[2.1, -0.3, 7.8, 0.05, -1.2, 4.9, 0.01, ...]
```

And one neuron computes:

```
output = 0.7 × 2.1   +  0.4 × (-0.3)  +  0.6 × 7.8  +  0.9 × 0.05  + ...
       =   w₁ × a₁   +   w₂ × a₂      +   w₃ × a₃   +   w₄ × a₄    + ...
```

Look at the contribution of each weight:

Weight	Activation paired with	Contribution	Salient?
$w_1 = 0.7$	$a_1 = 2.1$	1.47	Medium
$w_2 = 0.4$	$a_2 = -0.3$	−0.12	No
$w_3 = 0.6$	$a_3 = 7.8$	4.68	YES — large activation
$w_4 = 0.9$	$a_4 = 0.05$	0.045	No — tiny activation

If you round w_3 poorly (say, 0.6 → 0.5), the contribution shifts from 4.68 to 3.90. A loss of 0.78 — large.

If you round w_4 disastrously (0.9 → 1.5), the contribution changes from 0.045 to 0.075. A loss of 0.03 — invisible.

> A weight paired with a chronically large activation is salient. A weight paired with a chronically tiny activation is filler. You can crush filler weights aggressively and protect the salient ones.

The paradox: how do we know in advance?

Activation values depend on the input. So how can we know that w_3 will always be paired with a large activation, when different inputs produce different activations?

The answer is one of the most surprising empirical findings in modern LLM research:

> The activation dimensions themselves are loud or quiet, regardless of input. The same dimensions stay loud across virtually all inputs. The same dimensions stay quiet across virtually all inputs.

The emergent outlier channel phenomenon

Suppose layer 5's incoming activation vector has 4096 dimensions — call them positions 0 through 4095. Run the model on 500 different inputs (cats, France, quantum mechanics, anything). Record the magnitude of each position across all runs and average them:

```
Position    | Avg magnitude across 500 inputs
------------|-------------------------------
    0       |   0.4
    1       |   1.2
    2       |   0.3
   ...      |  ...
  2378      |  47.3   ← LOUD
   ...      |  ...
  3914      |  52.8   ← LOUD
   ...      |  ...
  4095      |   0.1
```

These rankings stay nearly identical across inputs. Position 2378 is loud whether the input is about cats or France. Position 4095 is quiet for everything.

Why? During training, the model learned to use specific dimensions as information highways. Position 2378 might encode something general like 'syntactic role' that every input has. Position 4095 might be scratch space the model never figured out how to use. These patterns are baked into the trained weights.

Salience is structural, not numerical

Now here's the key consequence. A weight matrix in layer 5 might be 4096 × 4096. Each output neuron has 4096 input weights, and each weight is paired with one fixed activation position:

```
output_neuron[42] = w[42, 0]    × a[0]
                  + w[42, 1]    × a[1]
                  + w[42, 2]    × a[2]
                  + ...
                  + w[42, 2378] × a[2378]   ← always pairs with the loud channel
                  + ...
                  + w[42, 4095] × a[4095]
```

The weight w[42, 2378] is salient because the position it sits in always carries a large activation. This is true for every weight in column 2378 across all output neurons. So salience is a column-level structural property — entire columns of the weight matrix are flagged as salient.

Empirically, only about 1% of columns correspond to loud activation channels. That means roughly 1% of weights are salient — and protecting just that 1% recovers most of the accuracy loss from quantization.

Calibration dataset

To find which positions are loud, you don't need real production data. You need a small calibration dataset — typically 128 to 512 representative text samples. Run them through the model, record activation magnitudes per position, average them. Done.

This is what 'activation-aware' in AWQ literally means: quantization decisions made aware of the activation patterns observed during calibration.

5. RTN — Round To Nearest

The strategy

RTN is the bucket trick from Section 2 applied uniformly. Find the range, slice into buckets, snap each weight to its closest bucket. No favoritism, no awareness of activations, no calibration data.

Concrete example

Take our 10 weights. RTN simply runs the bucket trick once and stores the slot numbers. That's the whole algorithm.

The flaw

RTN treats all weights equally. The salient 1% gets rounded just as carelessly as the boring 99%. When a salient weight has a small rounding error of +0.0368, and it gets multiplied by an activation of 5.2 every time a token passes through, the propagated error per inference is:

```
0.0368 × 5.2 = 0.19    per neuron, per layer, per token
```

Across 32 layers and millions of neurons, these errors compound. The model still runs, but quality drops.

When to use RTN

Scenario	RTN appropriate?
8-bit quantization (errors per weight are tiny anyway)	Yes — RTN at INT8 is fine
4-bit on a small model	Maybe — depends on tolerance
4-bit on a large model (7B+)	No — use GPTQ or AWQ
No calibration data available	RTN is your only option
Need fastest possible quantization	RTN runs in seconds

> RTN is the baseline. Every other method exists because RTN's accuracy degrades too much at low bit-widths.

6. GPTQ — Round With Error Compensation

GPTQ stands for Generative Pre-trained Transformer Quantization (named after the GPT family it was designed for).

The insight

> When I round one weight, I introduce an error. Why don't I let the not-yet-rounded weights nudge themselves slightly to cancel that error out?

Roommate analogy

You and three roommates split rent equally — $1000 total, $250 each. Your bank only accepts whole hundreds. You round your share to $300, overpaying by $50. Instead of just absorbing the loss, the other three each pay $250 − $16.67 = $233.33. Total stays at $1000.

That's GPTQ. Round one weight at a time. After each rounding, look at the error introduced and adjust the not-yet-rounded weights to compensate.

Worked example with 4 weights

Original weights: [0.738, 0.592, 0.337, −0.092]. Buckets are 0.1 wide.

Step	Action	Error	Effect on remaining weights
1	Round 0.738 → 0.7	+0.038	Distribute −0.013 to each of 3 remaining → [—, 0.579, 0.324, −0.105]
2	Round 0.579 → 0.6	−0.021	Distribute +0.011 to each of 2 remaining → [—, —, 0.335, −0.094]
3	Round 0.335 → 0.3	+0.035	Push to last remaining → [—, —, —, −0.129]
4	Round −0.129 → −0.1	—	Done

Total accumulated error across the layer is much smaller than RTN's because each error gets partially absorbed by neighbors instead of just sitting there.

The Hessian refinement

In practice, GPTQ doesn't distribute errors equally. It uses the Hessian matrix (computed from the calibration data) to figure out which neighbors can absorb errors with the least output disturbance. Some neighbors absorb more, some less, depending on their sensitivity.

The Hessian comes from second-order calculus on the loss — the same calculus involved in training. You can ignore the math; what matters is that the calibration data tells GPTQ how to distribute errors intelligently.

Trade-offs

Property	RTN	GPTQ
Accuracy at INT4	Drops noticeably	Near-original
Computation time	Seconds	Hours (for large models)
Calibration data needed	No	Yes (~128 samples)
Implementation complexity	Trivial	Significant

7. AWQ — Activation-aware Weight Quantization

The insight

> Instead of trying to round salient weights well, protect them from being rounded badly in the first place by scaling them up before they hit the bucket grid.

Volume knob analogy

You're recording someone whispering and someone shouting in the same room with one microphone that has only 16 volume levels. RTN sets the mic at one fixed level — the whisperer's voice gets lost in the noise floor. AWQ adds a 'boost' knob just for the whisperer — turns up their input volume, then turns it back down on playback. The whisper is recorded clearly without affecting the shouter.

How it works mechanically

The mathematical identity AWQ relies on:

```
weight × activation  =  (weight × s) × (activation / s)

for any scale factor s.
```

The product on both sides is identical. AWQ uses this freedom strategically:

Step	Action
1	Run calibration data through the model. Identify the ~1% loud activation channels.
2	For each salient weight column (paired with a loud channel), pick a scale factor s > 1.
3	Multiply the salient weights by s. This makes them larger relative to the bucket grid.
4	Quantize all weights using the bucket trick. Salient weights are now in a region of the grid where rounding errors are proportionally smaller.
5	At inference time, divide the corresponding activations by s to keep the math identical.

Concrete example

A salient weight of value 0.05, where buckets are 0.1 wide:

Approach	What happens	Recovered value	Error
RTN (no scaling)	0.05 → bucket 0 → recovered as 0.0	0.0	100% (catastrophic)
AWQ (scale by 8×)	0.05 × 8 = 0.4 → bucket 4 → recovered as 0.4 → ÷8 = 0.05	0.05	≈0%

Meanwhile non-salient weights are quantized normally and nobody cares about their rounding errors because their activations are tiny anyway.

Why AWQ wins in practice

Property	GPTQ	AWQ
Quantization speed	Hours (Hessian computation)	Minutes (just scale + round)
Accuracy at INT4	Excellent	Equal or slightly better
Inference speed	Standard	Often faster (cleaner kernel)
Calibration data needed	Yes	Yes
Adoption today	Older standard	Current default for production

> AWQ has become the default for 4-bit production deployments of Llama, Mistral, and most open-source LLMs as of 2024–2026. The combination of fast quantization, excellent accuracy, and clean inference kernels makes it the practical winner.

8. Side-by-Side Method Summary

Method	Strategy	Computation cost	Accuracy at 4-bit	When to use
RTN	Snap each weight to nearest bucket. Done.	Seconds	Lowest	Quick experiments, INT8 (where errors are tiny anyway)
GPTQ	Round one weight at a time. Compensate using Hessian-weighted error distribution.	Hours	Excellent	When you can wait and want highest accuracy
AWQ	Identify the 1% salient weights. Scale them up before rounding so their errors shrink, scale activations down at runtime.	Minutes	Excellent	Default choice for 4-bit production today

The progression

- RTN ignores importance and treats all weights equally.
- GPTQ corrects errors after the fact by adjusting neighboring weights.
- AWQ prevents errors on important weights in the first place by scaling them into a forgiving region of the bucket grid.

9. Why Quantization Doesn't Break the Model

It's surprising that rounding billions of weights doesn't destroy accuracy. Three reasons it mostly survives:

1. Neural networks are noisy by design

Models are trained with dropout, weight decay, and stochastic gradient noise. They've already learned to be robust to small perturbations in their weights. Quantization noise is just one more perturbation in a system that's been hardened against perturbations.

2. Errors cancel out

When a neuron sums hundreds of (weight × activation) products, the rounding errors are roughly random — some positive, some negative. The variance averages down. Across hundreds of contributions per neuron, the per-neuron error is much smaller than the per-weight error.

3. Smarter quantization protects what matters

AWQ's headline result: a well-quantized 4-bit Llama-70B keeps roughly 99% of original FP16 accuracy on standard benchmarks while using 4× less memory. That's why every open-source model now ships in quantized form.

10. Weight vs Activation Quantization

So far we've been talking about weight quantization — making the stored weights smaller. There's a separate topic: activation quantization — quantizing the live numbers flowing through the network at inference time.

	Weight quantization	Activation quantization
What gets quantized	Stored weights in the model file	Numbers computed at inference time
Primary benefit	Smaller model size	Faster inference (integer math is cheap)
Difficulty	Manageable	Hard — outlier channels destroy precision
Common in production?	Yes (almost universal)	Sometimes (often skipped or done carefully)

Activation quantization is hard precisely because of those outlier channels we discussed. If one channel has values up to 100 and another has values up to 0.5, quantizing them with the same bucket grid

destroys the smaller channel. Many production systems quantize weights aggressively to INT4 but keep activations in FP16 to avoid this problem.

11. Key Vocabulary

Term	Meaning
FP32 / FP16	32-bit / 16-bit floating point. The default precision for trained weights.
INT8 / INT4	8-bit / 4-bit integer. The target precision after quantization.
Scale factor	The size of one bucket. Maps integer slot numbers back to approximate floats.
Zero point	Where the value 0.0 lands in the bucket scheme. Zero is special because it appears often in neural networks.
Bucket / slot / level	Synonyms — one of the discrete values an integer quantization can represent.
Calibration dataset	Small set of representative inputs (typically 128–512 samples) used to identify activation patterns.
Salient weight	A weight paired with a chronically large activation. Rounding it badly significantly hurts model output.
Outlier channel	An activation dimension that consistently carries large values across nearly all inputs.
PTQ	Post-Training Quantization. Quantizing a fully-trained model after training is done. RTN, GPTQ, AWQ all fall here.
QAT	Quantization-Aware Training. Training the model knowing it'll be quantized later. Higher cost, sometimes higher quality.
RTN	Round To Nearest. The naive baseline.
GPTQ	Generative Pre-trained Transformer Quantization. Hessian-based error compensation.
AWQ	Activation-aware Weight Quantization. Salient-weight scaling.

12. Quick Reference — When to Use What

Goal	Recommended approach
Run a 70B model on a single 80GB GPU	INT4 with AWQ
Get max accuracy, can wait hours	INT4 with GPTQ
Quick proof-of-concept, no calibration data	INT8 with RTN

Goal	Recommended approach
Run on CPU or edge device	INT4 weight quant + INT8 activation quant
Can't lose any accuracy	FP16 (don't quantize) — accept the memory cost
Fine-tuning a quantized model	QLoRA — combines INT4 weights with LoRA adapters

> In 2026, the practical default for deploying open-source LLMs is: load the AWQ-quantized INT4 version. Performance and quality have converged to the point where there's rarely a reason not to.